UniGrant: Research Grant Lifecycle Management

Final database design document

Chaitrali Shrikant Dharmadhikari (2024A7PS0133H) · Maddela Bhargava Krishna Reddy (2024A8PS0349H) · Lakshya Sachdeva (2024A8PS0468H) · Risha Sinha (2024A8PS0485H) · Arnav Gupta (2024A8PS0487H)

BITS Pilani, Hyderabad Campus · November 2026 · https://github.com/BinaryCoder3012/DBMS-Project

1 Problem, Users and Assumptions

UniGrant keeps the whole life of a university research grant in one MySQL 8.0 database: proposal, review, award, projects, budget heads, expenses, milestones and research outputs. The rules that matter most are enforced by the DBMS itself through constraints and triggers, so they hold for every client, not only for this application.

Role	Who	Needs	DB role
Admin	Research office	University-wide analytics, user management	admin_role
PI	Faculty	Submit proposals, log expenses, update milestones	pi_role
Grant Officer	Finance, compliance	Review proposals, award grants, approve or reject expenses	grant_officer_role
Student	Research assistants	Read assigned projects, no financial data	student_role

Assumptions. (A1) A user has one role and at most one department. (A2) A project has exactly one PI and at most one grant; a grant may fund several projects. (A3) Money is DECIMAL(15,2) in rupees, single currency. (A4) Only Approved expenses count towards spent_amount. (A5) MySQL has no row-level security policy, so row filtering is done with views and the application layer.

Business rules and enforcement

ID	Rule	Enforcement
BR1	An expense cannot exceed the remaining balance of its budget head	Trigger balance_check_before_expense; row locks and chk_budget_amounts at approval
BR2	Approval moves the amount into spent_amount atomically; reversal restores it	Trigger update_budget_on_approval in a transaction
BR3	A PI with 5 active projects cannot propose	Trigger active_project_limit_pi
BR4	A receipt reference is unique within a project	Trigger prevent_duplicate_expense_receipt
BR5	Dates are ordered, amounts positive, no self-supervision	11 CHECK constraints
BR6	Students see only assigned projects and no financial columns	Views Student_Project_View, Public_Project_View; API role guard
BR7	Grant officers can never create users or alter the schema	REVOKE CREATE USER; least-privilege GRANTs
BR8	Expired grants close; milestones due in 30 days notify the PI	Events grant_closure_check, milestone_30day_alert

2 Application Design

Architecture. Three tiers: React 19 and Vite 8 single-page app, Express 4 REST API with mysql2 (pooled, parameterised queries), MySQL 8.0 on InnoDB. Business rules live in the data tier. The API authenticates callers with signed session tokens, applies the role guard and row filters, runs multi-statement transactions, and maps database errors to HTTP responses, so SQLSTATE 45000 reaches the user as a 422 with the database's message.

Workflow	What happens	Demonstrates
W1 Award a grant	An officer awards a proposal. One transaction approves it and inserts Grant_, Project and BudgetHead rows. Any failure rolls all of them back.	Multi-table transaction
W2 Expense	A PI logs an expense; triggers refuse an overspend or duplicate receipt. Approval locks the rows, and the AFTER UPDATE trigger moves the money in the same transaction.	Integrity enforcement, concurrency
W3 Discover and report	FULLTEXT researcher and call matching, views, a stored-procedure report and queries Q1 to Q6.	Complex retrieval

3 ER / EER Diagram (Chen Notation)

The model has 12 strong entities, 3 weak or associative entities with composite keys (Dependent, Equipment, PublicationAuthor), 3 M:N relationships with attributes (WORKS_ON, MATCHES, recursive SUPERVISES) and one specialisation of User by role (Admin, PI, Grant Officer, Student; disjoint, total), mapped to one table with a discriminator. BudgetHead and Milestone are existence-dependent on Project. Rectangles are entities, double rectangles weak entities, diamonds relationships, double diamonds identifying relationships; 1, N and M are cardinalities.

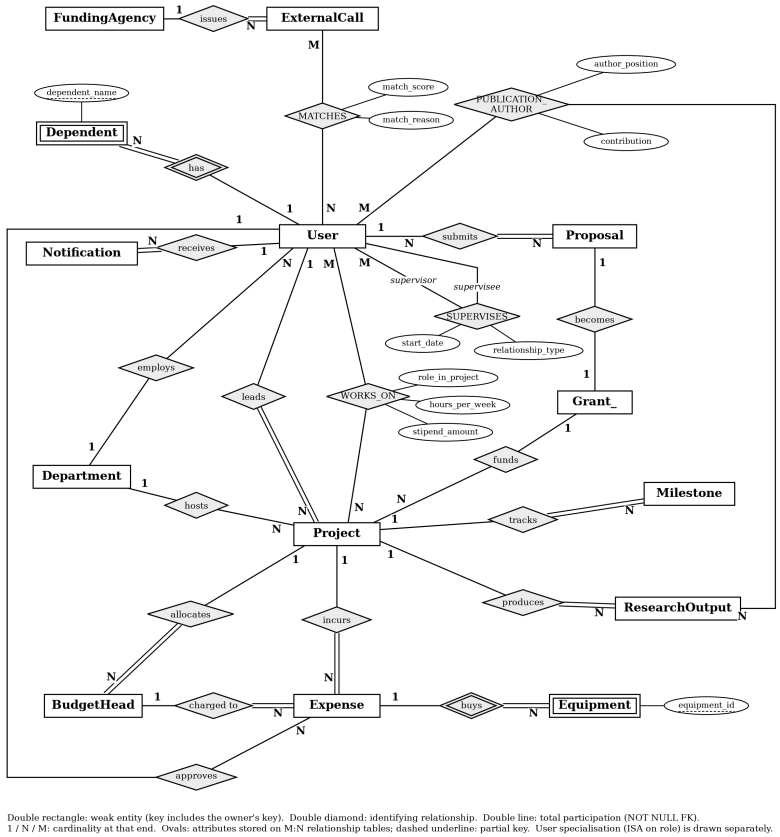


Fig. 1. Chen-style EER diagram of the complete schema.

Relationship	Participants	Card.	Participation and attributes
WORKS_ON	User, Project	M:N	role in project, hours per week, stipend
MATCHES	ExternalCall, User	M:N	match score 0 to 100, reason, computed at
SUPERVISES	User, User	M:N	recursive; relationship type, start date
PUBLICATION_AUTHOR	User, ResearchOutput	M:N	author position, contribution
becomes	Proposal, Grant_	1:0..1	Grant partial; UNIQUE (proposal_id)
funds	Grant_, Project	1:N	Project partial; one grant may fund several projects (A2)
leads	User (PI), Project	1:N	Project total (pi_id NOT NULL, RESTRICT)
allocates	Project, BudgetHead	1:N	BudgetHead existence-dependent (CASCADE)
charged to	BudgetHead, Expense	1:N	Expense total (RESTRICT protects financial history)
approves	User, Expense	1:N	Expense partial; SET NULL keeps the expense if the approver leaves
buys	Expense, Equipment	1:N	Equipment weak, key (equipment_id, expense_id)
has	User, Dependent	1:N	Dependent weak, key (user_id, dependent_name)

4 Relational Schema

Mapping rules: strong entity to one relation; weak entity to a relation whose key includes the owner key; 1:N to a foreign key on the N side; M:N to a bridge relation with a composite key; specialisation to a single table with a discriminator; derived value to a stored generated column. Underlined = primary key, *italic* = foreign key. 18 relations plus the archive BudgetSnapshot, written by a scheduled event, make 19.

Relation	Attributes
Department	dept_id, dept_name, location, annual_budget, established_date
User	user_id, ssn (UQ), first_name, middle_initial, last_name, email (UQ), password_hash, role, phone, address, date_of_birth, hire_date, specialization, *dept_id*
FundingAgency	agency_id, agency_name, contact_person, email, phone, address, website, funding_focus_areas
ExternalCall	call_id, title, description, eligibility_criteria, deadline, max_budget, *agency_id*
Proposal	proposal_id, title, abstract, total_budget_requested, duration_months, submission_date, status, eligibility_check_passed, *pi_id*
Grant_	grant_id, grant_title, total_amount_awarded, start_date, end_date, status, funding_agency, agency_contact_info, reporting_frequency, *proposal_id* (UQ)
Project	project_id, project_title, description, start_date, end_date, status, current_balance, *department_id*, *grant_id*, *pi_id*
BudgetHead	budgethead_id, *project_id*, category, allocated_amount, spent_amount, remaining_balance (generated), fiscal_year
Expense	expense_id, amount, date, description, receipt_reference, status, *budgethead_id*, *project_id*, approval_date, *approved_by*
Milestone	milestone_id, *project_id*, title, due_date, completion_date, deliverable_description, status, report_submitted, priority
ResearchOutput	output_id, *project_id*, type, title, authors, publication_date, journal_conference, doi, status
Notification	notif_id, *user_id*, message, is_read, sent_at
Dependent	*user_id*, dependent_name, relationship, date_of_birth, phone
PublicationAuthor	*output_id*, *user_id*, author_position, contribution_description
Equipment	equipment_id, *expense_id*, equipment_name, serial_number, warranty_until, location
WORKS_ON	*user_id*, *project_id*, role_in_project, hours_per_week, stipend_amount
SUPERVISES	*supervisor_id*, *supervisee_id*, relationship_type, start_date
MATCHES	*call_id*, *user_id*, match_score, match_reason, notification_sent, computed_at
BudgetSnapshot	snapshot_id, snapshot_date, budgethead_id, project_id, category, allocated_amount, spent_amount, remaining_balance

Integrity constraints and referential actions

Referential actions. RESTRICT protects financial history: Proposal.pi_id, Project.pi_id, Expense.budgethead_id. SET NULL keeps optional links harmless: User.dept_id, Grant_.proposal_id, Project.department_id, Project.grant_id, Expense.approved_by. CASCADE removes existence-dependent rows: budget heads, milestones, outputs, notifications, dependents, equipment and all bridge rows.

Constraints. DECIMAL(15,2) for all money; ENUM for every status and category; UNIQUE on User.email, User.ssn and Grant_.proposal_id; NOT NULL on mandatory keys and amounts. 11 CHECK constraints: positive requested budget, duration, grant amount, expense amount and allocation; end_date > start_date on Grant_ and Project; current_balance >= 0; 0 <= spent_amount <= allocated_amount; supervisor_id != supervisee_id; match_score BETWEEN 0 AND 100; role domain. remaining_balance = allocated_amount - spent_amount is a STORED generated column. The grant relation is named Grant_ because GRANT is reserved.

Foreign key	References	ON DELETE	Foreign key	References	ON DELETE
User.dept_id	Department	SET NULL	Expense.budgethead_id	BudgetHead	RESTRICT
ExternalCall.agency_id	FundingAgency	CASCADE	Expense.project_id	Project	CASCADE
Proposal.pi_id	User	RESTRICT	Expense.approved_by	User	SET NULL
Grant_.proposal_id	Proposal	SET NULL	Milestone.project_id	Project	CASCADE
Project.department_id	Department	SET NULL	ResearchOutput.project_id	Project	CASCADE
Project.grant_id	Grant_	SET NULL	Notification.user_id	User	CASCADE
Project.pi_id	User	RESTRICT	Equipment.expense_id	Expense	CASCADE
BudgetHead.project_id	Project	CASCADE	Dependent.user_id	User	CASCADE

5 Normalisation and Design Justification

All relations are in 3NF. Every determinant is a candidate key except for the redundancy noted for Expense, so the others are also in BCNF.

Relation	Key dependencies	Observation	NF
User	user_id → all; email → user_id; ssn → user_id	Candidate keys user_id, email, ssn	BCNF
BudgetHead	budgethead_id → all; (allocated, spent) → remaining	Derived value is a generated column, so it cannot disagree	BCNF
Expense	expense_id → all; budgethead_id → project_id	project_id is derivable through budgethead_id; kept for fast project queries	3NF
WORKS_ON, Dependent, Equipment	composite key → all non-key attributes	No partial dependency	BCNF

Documented denormalisations. (D1) Grant_.funding_agency is a snapshot of the agency name at award time. (D2) Project.current_balance is a cached roll-up maintained by update_budget_on_approval in the approval transaction; a test asserts that it equals the sum of the head balances. (D3) ResearchOutput.authors is a display string; ordered authorship lives in PublicationAuthor.

Important schema-design decisions

Decision	Reason
Rules live in the DBMS	Triggers and CHECKs hold for every client. A direct INSERT is refused exactly as an API request is.
Single-table ISA for User	The four roles differ in privileges, not stored attributes, so subclass tables would hold no columns.
Surrogate integer keys	They keep foreign keys narrow and stable. Weak entities keep composite keys that include the owner.
STORED generated column	It removes an update anomaly, and unlike a VIRTUAL column it is materialised for reads.
auto_close_project is BEFORE UPDATE	A trigger cannot update its own table (error 1442), so it assigns NEW.status.
spent_amount <= allocated_amount as a CHECK	Pending expenses do not reduce the balance, so two can each pass the insert trigger and overspend together. The API locks rows at approval, and the CHECK is the backstop for any other client.
Views pre-aggregate per department	Joining several one-to-many tables in one FROM clause multiplies rows and inflates SUM. Each measure is aggregated in its own derived table first.
Student view exposes student_id	The API uses a pooled service account, so CURRENT_USER() cannot identify a person. The view omits money columns, and the API filters by the identity in the signed token (A5).

6 DBMS Features Implemented

Feature	Objects and purpose
Views (4)	Public_Project_View (hides money), PI_Dashboard_View (per-project aggregates), Admin_Analytics_View (department roll-up), Student_Project_View (assignments, no money)
Indexes (6)	B-tree (project_title, status), (last_name, first_name), (date, project_id), (status, due_date); FULLTEXT on Proposal (abstract, title) and User (specialization); checked with EXPLAIN
Triggers (5)	BR1 to BR4 and auto_close_project
Transactions	Grant award over four tables; expense approval with SELECT ... FOR UPDATE
Procedures (2), events (3)	generate_annual_report, get_pi_budget_summary; milestone_30day_alert, monthly_budget_snapshot, grant_closure_check
Roles (4)	admin, pi, grant_officer and student with least-privilege GRANTs; students reach data only through views
Queries (6)	Q1 department utilisation (join, GROUP BY, HAVING); Q2 above-category-average expenses (correlated subquery); Q3 top 3 projects per department (CTE, RANK() OVER); Q4 active projects with no recent milestone (NOT EXISTS); Q5 researcher matching (FULLTEXT); Q6 supervision chain (recursive CTE)

Data and verification. The seed holds 1,335 records across 17 populated relations; approved balances are produced by the triggers, not typed in. MATCHES and BudgetSnapshot fill at run time. A suite of 32 integration tests runs on MySQL 8.0.46, covering the rules above, rollback, RBAC with real database accounts, and concurrent approvals.